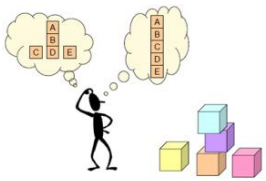
Unidad 1: Tipos de Datos Fundamentales. Arreglos – Registros

- **Almacenamiento de datos.** Definición. Estructura de datos. Clasificación.
- **Arreglos.** Definición. Clasificación. Operaciones. Definición formal. Implementación en memoria principal.
- **Registros.** Definición. Componentes. Operaciones. Almacenamiento. Usos. Registros vs. Arreglos. Combinación entre registros y arreglos. Registros Lógicos. Registros Físicos.
- Búsqueda Interna y Ordenamiento Interno de datos.

Búsqueda y Ordenamiento

- **Búsqueda**.
  - Es el proceso de localizar un dato.
  - Se puede llevar a cabo sobre elementos ordenados o desordenados.
  - Si los elementos están ordenados, la búsqueda es más fácil y más rápida.
  - Los métodos de búsqueda pueden ser:
    - **Internos**: Los elementos se encuentran en MP
    - **Externos**: Los elementos se encuentran en MS

- **Ordenamiento**
  - Ordenar significa reagrupar o reorganizar un conjunto de datos u objetos en una secuencia específica.
  - Puede ser:
    - **Interno**: Los elementos se encuentran en MP
    - **Externo**: Los elementos se encuentran en MS



Búsqueda interna

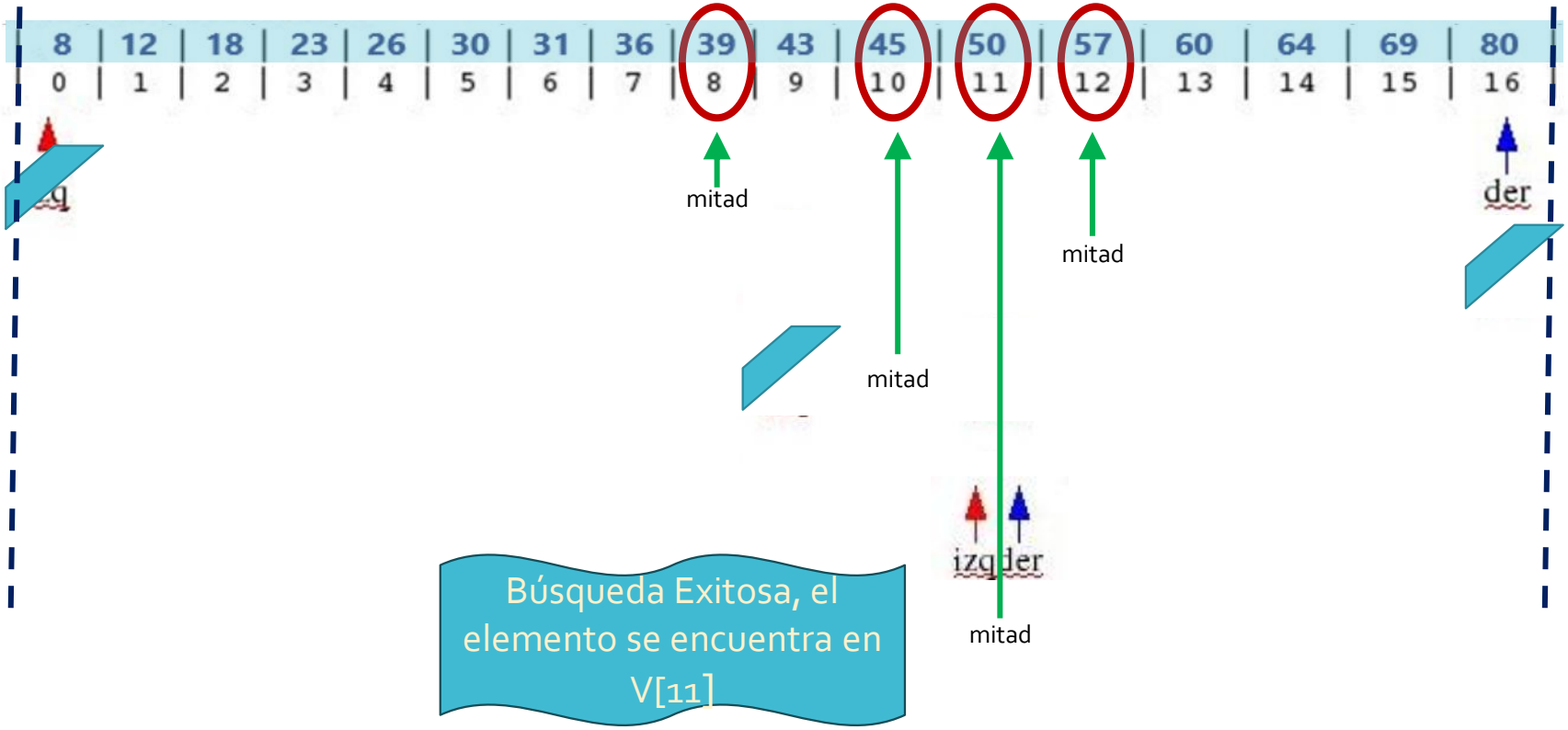
- Los métodos de búsqueda interna más importantes son:
  - Secuencial o lineal
  - Binaria
  - Por transformación de claves
  - Arboles de búsqueda

BÚSQUEDA BINARIA

- Consiste en dividir el intervalo de búsqueda en dos partes, comparando el elemento buscado con el que ocupa la posición central en el arreglo.
  - Si los elementos no fueran iguales, se redefinen los extremos del intervalo, según el elemento central sea mayor o menor que el buscado.
  - El proceso concluye cuando:
    - El elemento es encontrado
    - El intervalo de búsqueda se anula, es vacío

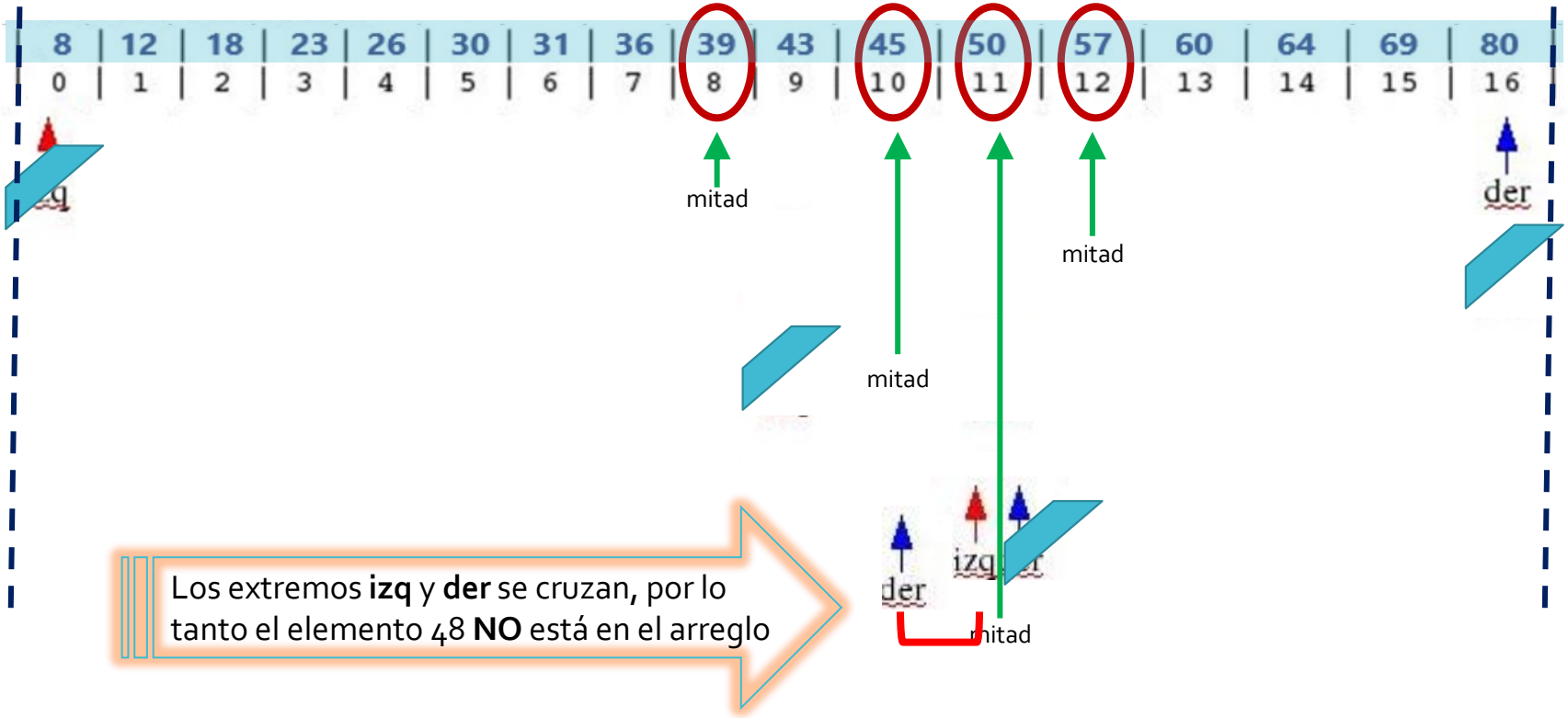
BUSQUEDA BINARIA

Sean: el arreglo **V** de valores enteros, ordenado ascendentemente, **buscar** el elemento **50**



BUSQUEDA BINARIA

Sean: el arreglo **V** de valores enteros, ordenado ascendentemente, **buscar** el elemento **48**



Búsqueda por transformación de claves

- Permite aumentar la velocidad de búsqueda sin necesidad de tener los elementos ordenados. El tiempo de búsqueda es independiente del número de componentes del arreglo.

- Cada dato tiene asociado una clave. Para encontrarlo directamente por su clave el método trabaja utilizando una función hash que convierte una clave dada en una dirección –índice- dentro del arreglo.

$$\text{dirección} \leftarrow H(\text{clave})$$

- Se requiere:
  - Una función hash que sea fácil de calcular y distribuya uniformemente las claves
  - Un método para resolver colisiones

Búsqueda por transformación de claves

- **Función hash por módulo**: división

$$H(K) = (K \bmod N) + 1$$

- Se recomienda que N sea un número primo o divisible entre muy pocos números. Si N no es primo, se debe considerar el valor primo más cercano.

- Ejemplo:
  - Sea N=100 el tamaño del arreglo y sean sus direcciones los números entre 1 y 100.
  - Sean k1=7259 y k2=9357 dos claves a las que deban asignarse posiciones en el arreglo.
  - H(k1)=7259 mod 100 +1 =60
  - H(k2)=9359 mod 100 +1 =60

- Solución a la colisión.
  - Aplicar función de reasignación rh(I) donde I es la dirección colisionada:
  - rh(i)=(I+1) mod 100
  - rh(i)=(60+1) mod 100 =61

Búsqueda por transformación de claves

- Para trabajar con el método de búsqueda debe elegirse previamente:
  1. Una función Hash que sea fácil de calcular y que distribuya uniformemente las claves.
  2. Un método para resolver colisiones. Si éstas se presentan se debe contar con algún método que genere posiciones alternas.

Búsqueda por transformación de claves

FUNCIÓN CUADRADO

- Consiste en elevar al cuadrado la clave y tomar los dígitos centrales como dirección. El número de dígitos a tomar queda determinado por el rango del índice.
- Sea K la clave del dato a buscar.

  H(k)=digitos_centrales(k^2)+1

- La suma de una unidad a los dígitos centrales es para obtener un valor entre 1 y N.

- Ejemplo:
  - Sea N=100 el tamaño del arreglo y sus direcciones entre 1 y 100
  - Sea K1= 7259 y K2=9359, Dos claves a los que deben asignarse posiciones en el arreglo.

  - K1 al cuadrado = 52693081
  - K2 al cuadrado= 87590881
  - Por lo tanto: H(K1)=digitos_centrales (526 93 081) +1= 94
  - H(K2)=digitos_centrales (875 90 881) +1= 91

Búsqueda por transformación de claves

- **Prueba lineal**
  - Consiste en que una vez detectada la colisión se debe recorre el vector secuencialmente a partir del punto de detección hasta el elemento.
  - El proceso de búsqueda concluye cuando el elemento es hallado o cuando se encuentra una posición vacía. Se trata al arreglo como una estructura circular. El siguiente elemento después del último es el primero.

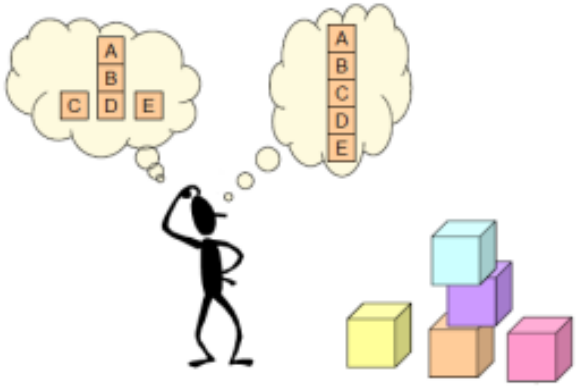
- **Encadenamiento**
  - Consiste en que cada elemento del arreglo tenga un apuntador a una lista ligada, la cual se irá generando e irá almacenando los valores colisionados a medida que se requiera. Es el método más eficiente debido al dinamismo propio de las listas.

Análisis de búsqueda

	C_{min}	C_{med}	$C_{máx}$
Búsqueda Secuencial	1	(1+N)/2	N
Búsqueda Binaria	1	$(1+\log_2(N))/2$	$\text{Log}_2(N)$

Ordenamiento

- ***Ordenar*** es:
- *"proceso de reorganizar un conjunto dado de objetos en una secuencia determinada".*
  - Una lista de datos está ordenada por un valor e si la lista **está en orden** con respecto al valor anterior.
  - Este Orden puede ser:
    - Ascendente: $e_i <= e_{i+1}$
    - Descendente: $e_i >= e_{i+1}$



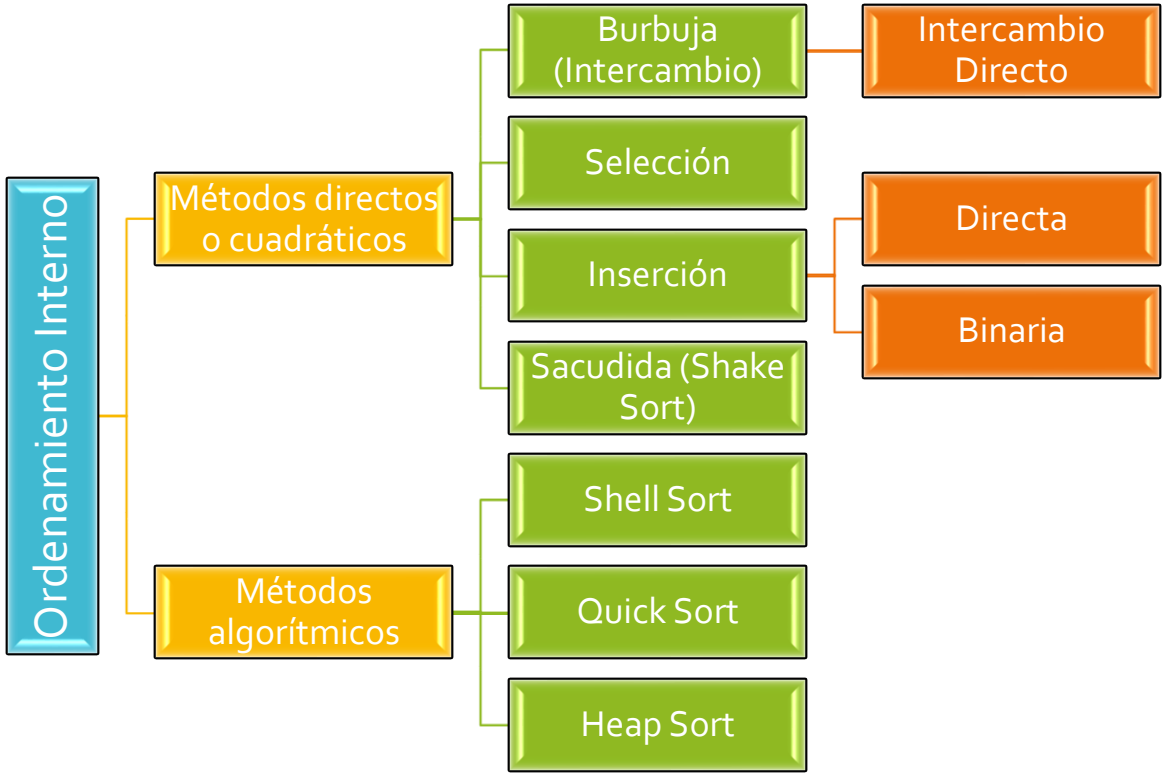
Ordenamiento

- Si consideramos un vector vec
- En el orden **ASCENDENTE,** se irán acomodando los elementos tal que los mayores quedarán hacia la derecha y los menores a la izquierda, de esta manera se observarán los elementos de menor a mayor. Es decir **vec[i]<=vec[i+1]**
- En el orden **DESCENDENTE,** los elementos más grandes se ubicarán a la izquierda y los más pequeños se irán ubicando hacia la derecha. Es decir, **vec[i]>=vec[i+1]**

Ordenamiento: Tipos

Internos	Recibe este nombre ya que los elementos o componentes del arreglo se encuentran en la **memoria principal** de la computadora.
Externos	Se aplican a grandes volúmenes de datos, que residen parcial o totalmente en dispositivos de **almacenamiento secundario**, tales como los discos. Aquí, el tiempo de acceso de lectura y escritura influye en la determinación de la eficiencia del ordenamiento.

Ordenamiento



Criterios de Eficiencia

- El **número de pasos**.
- El **número de comparaciones** entre valores para ordenarlos. Se utiliza cuando la comparación entre valores es costosa.
- El **número de movimientos** o **intercambios** de valores que se requieren para ordenar n valores. Se usa cuando el movimiento de valores es costoso

son los más simples y fáciles de entender, son eficientes cuando se trata de una cantidad pequeño de datos

Métodos directos o cuadráticos (n^2)

Ordenamiento Interno

Método SHAKER SORT (de la Sacudida)

- Es una **optimización** del **método de intercambio directo** (burbuja)
- Consiste en mezclar las dos formas del método de la burbuja
- Cada pasada tiene dos etapas:
  - 1ª Etapa: de **derecha a izquierda**, se mueven los elementos más pequeños hacia la parte izquierda del arreglo, almacenando en una variable la posición del último elemento intercambiado.
  - 2ª Etapa: de **izquierda a derecha**, se mueven los elementos más grandes hacia la parte derecha del arreglo, almacenando en otra variable la posición del último elemento intercambiado.
- Las sucesivas pasadas trabajan con los componentes del arreglo comprendidos entre las posiciones almacenadas en las vbles auxiliares.
- El algoritmo termina cuando en una etapa no se producen intercambios, o bien cuando el contenido de la variable que guarda el extremo izquierdo del arreglo es mayor que el contenido de la variable que almacena el extremo derecho.

SHAKER SORT: Primera pasada

- **PRIMERA ETAPA:**
  - **De derecha a izquierda :**
  - Se trasladan los elementos más pequeños hacia la parte izquierda del arreglo, almacenando en una variable la posición del último elemento intercambiado

	0	1	2	3	4	5	6	7
A:	15	67	08	16	44	27	12	35

PRIMERA PASADA

Primera etapa (de derecha a izquierda):

A[6] > A[7] (12>35) no hay intercambio

A[5] > A[6] (27>12) **si** hay intercambio

A[4] > A[5] (44>12) **si** hay intercambio

A[3] > A[4] (16>12) **si** hay intercambio

A[2] > A[3] (08>12) no hay intercambio

A[1] > A[2] (67>08) **si** hay intercambio

A[0] > A[1] (15>08) **si** hay intercambio

15	67	08	16	44	27	12	35
15	67	08	16	44	12	27	35
15	67	08	16	12	44	27	35
15	67	08	12	16	44	27	35
15	67	08	12	16	44	27	35
15	08	67	12	16	44	27	35
08	15	67	12	16	44	27	35

La última posición del intercambio de derecha a izquierda es 1.

Luego de la primera etapa de la primera pasada, el arreglo queda de la siguiente forma:

	0	1	2	3	4	5	6	7
A:	**08**	15	67	12	16	44	27	35

SHAKER SORT: Primera pasada

- **SEGUNDA ETAPA:**
  - **De izquierda a derecha:**
  - Se trasladan los elementos más grandes hacia la parte derecha del arreglo, almacenando en otra variable la posición del último elemento intercambiado.

	0	1	2	3	4	5	6	7
A:	**08**	15	67	12	16	44	27	35

Segunda etapa (de izquierda a derecha):

A[1] > A[2] (15>67) no hay intercambio

A[2] > A[3] (67>12) **si** hay intercambio

A[3] > A[4] (67>16) **si** hay intercambio

A[4] > A[5] (67>44) **si** hay intercambio

A[5] > A[6] (67>27) **si** hay intercambio

A[6] > A[7] (67>35) **si** hay intercambio

08	15	67	12	16	44	27	35
08	15	12	67	16	44	27	35
08	15	12	16	67	44	27	35
08	15	12	16	44	67	27	35
08	15	12	16	44	27	67	35
08	15	12	16	44	27	35	67

Última posición de intercambio de izquierda a derecha: 7.

Luego de la segunda etapa de la primera pasada, el arreglo queda de la siguiente forma:

	0	1	2	3	4	5	6	7
A:	**08**	15	12	16	44	27	35	**67**

	0	1	2	3	4	5	6	7
A:	**08**	15	12	16	44	27	35	**67**

Primera etapa (de derecha a izquierda):

A[5] > A[6] (27>35) no hay intercambio

A[4] > A[5] (44>27) **si** hay intercambio

A[3] > A[4] (16>27) no hay intercambio

A[2] > A[3] (12>16) no hay intercambio

A[1] > A[2] (15>12) **si** hay intercambio

08	15	12	16	44	27	35	67
08	15	12	16	27	44	35	67
08	15	12	16	27	44	35	67
08	15	12	16	27	44	35	67
08	12	15	16	27	44	35	67

Última posición de intercambio de derecha a izquierda: 2

A:	**08**	**12**	15	16	27	44	35	**67**
	0	1	2	3	4	5	6	7

	0	1	2	3	4	5	6	7
A:	**08**	**12**	15	16	27	44	35	**67**

Segunda etapa (de izquierda a derecha):

A[2] > A[3] (15>16) no hay intercambio

A[3] > A[4] (16>27) no hay intercambio

A[4] > A[5] (27>44) no hay intercambio

A[5] > A[6] (44>35) **si** hay intercambio

08	12	15	16	27	44	35	67
08	12	15	16	27	44	35	67
08	12	15	16	27	44	35	67
08	12	15	16	27	35	44	67

Última posición de intercambio de izquierda a derecha: 6.

	0	1	2	3	4	5	6	7
A:	**08**	**12**	15	16	27	35	**44**	**67**

	0	1	2	3	4	5	6	7
A:	**08**	**12**	15	16	27	35	**44**	**67**

Primera etapa (de derecha a izquierda)**:**

A[4] > A[5] (27>35) no hay intercambio

A[3] > A[4] (16>27) no hay intercambio

A[2] > A[3] (15>16) no hay intercambio

08	12	15	16	27	35	44	67
08	12	15	16	27	35	44	67
08	12	15	16	27	35	44	67

No se producen intercambios: FIN DEL ORDENAMIENTO.

A:	**08**	**12**	**15**	**16**	**27**	**35**	**44**	**67**
	0	1	2	3	4	5	6	7

SHAKER SORT: Rendimiento

Caso óptimo	Caso desfavorable	Caso medio
Los elementos se encuentran ordenados, se realizan n comparaciones	*Los elementos se encuentran en orden inverso*	*Los elementos se encuentran desordenados aleatoriamente*
$O(n)$	$O(n^2)$	$O(n^2)$

Método INSERCION BINARIA

Es una **mejora** del método de inserción directa. La mejora consiste en realizar una **búsqueda binaria** en lugar de una búsqueda secuencial, **para insertar** un elemento en la parte izquierda del arreglo, que ya se encuentra ordenado

```
void insercion_binaria(int n,float a[max]){
 int i,j,p,u,c;
 float aux;
 for (i=1;i<n;i++){
   aux=a[i];
   p=0;
   u=i-1;
   while(p<=u){
     c=(p+u)/2;
     if (a[c]>aux)  u=c-1;
     else p=c+1;
   }
   for(j=i-1;j>=p,j--)
     a[j+1]=a[j];
   a[p]=aux;
 }
}
```

Estado inicial	15	67	08	16	44	27	12	35

subvector ordenado

Elementos a ordenar

0	1	2	3	4	5	6	7
15	67	8	16	44	27	12	35

15	67

n	i	j	p	u	c	a[i]	aux	a[c]	a[p]
8	1		0	0	0	67	67	15	

INSERCIÓN BINARIA

Primera Comparación

Segunda Comparación

Tercera Comparación

Estado inicial	**15**	67	08	16	44	27	12	35
	15	**67**	08					
	08	15	67	16				
	08	15	**16**	67	44			
	08	15	16	**44**	67	27		
	08	15	16	**27**	44	67	12	
	08	**12**	15	16	27	44	67	35
	08	12	15	16	27	**35**	44	67

INSERCIÓN BINARIA: Rendimiento

- Al analizar el método de ordenación por inserción binaria se advierte la presencia de un caso antinatural. El método efectúa el menor número de comparaciones cuando el arreglo está totalmente desordenado y el máximo cuando se encuentra ordenado.

Antinatural. Si el arreglo está ordenado va a hacer más comparaciones.

- El número promedio de comparaciones es:

$$C = \frac{(n^2 - n)}{4}$$

son más complejos, difíciles de entender y son eficientes en grandes cantidades de datos.

Métodos Logarítmicos (n log n)

Ordenamiento Interno

Método SHELL

- Es una versión mejorada del método de **inserción directa**.
- También se conoce como **inserción con incrementos decrecientes.**
- En el método de **inserción directa** cada elemento se compara para su ubicación correcta en el arreglo con los elementos que se encuentran en su **parte izquierda**. Si el elemento a insertar es más pequeño que el grupo de elementos que se encuentran a su izquierda será necesario efectuar varias comparaciones antes de su ubicación.
- **Shell** propone que las **comparaciones** entre elementos se efectúen con **saltos** de mayor tamaño, pero con **incrementos decrecientes**; así, los elementos quedarán ordenados en el arreglo más rápidamente.

Estado Inicial

15	67	8	16	44	27	12	35

n=8

		27				67		
1° Pasada	15	~~67~~	8	16	44	~~27~~	12	35

salto= n/2=4

Intercambio	15	**27**	08	16	44	**67**	12	35

No hay intercambio

Se modifica salto

Luego de la 1° pasada

15	27	8	16	44	67	12	35

salto=4

	8	16	15	27	12	35	44	67
2° Pasada	~~15~~	~~27~~	~~8~~	~~16~~	~~44~~	~~67~~	~~12~~	~~35~~

salto= n/2=2

			12		15			
Intercambio	8	16	~~15~~	27	~~12~~	35	44	67

Intercambio	8	16	12	27	15	35	44	67

No hay intercambio

Se modifica salto

Luego de la 2°pasada

8	16	12	27	15	35	44	67

salto=2

		12	16	15	27			
3° Pasada	8	~~16~~	~~12~~	~~27~~	~~15~~	35	44	67

salto= salto/2=1

			15	16				
Intercambio	8	12	~~16~~	~~15~~	27	35	44	67

Intercambio	8	12	15	16	27	35	44	67

No hay intercambio

salto = 1
FIN

SHELL: Análisis de eficiencia

- El **tiempo de ejecución** del algoritmo es del orden de $n*(\log n)^2$.
- El **número de comparaciones** y de **movimientos** depende de la secuencia de intervalos (salto) que se escoja.

Método QUICKSORT

- Es actualmente el más eficiente y veloz de los métodos de **ordenación interna**. Es también conocido como **método rápido** y de **ordenación por partición**.
- La idea central de este algoritmo consiste en lo siguiente:
  - Se toma un elemento X de una posición cualquiera del arreglo
  - Se trata de ubicar a X en la **posición correcta** del arreglo, de tal forma que todos los elementos que se encuentren a su **izquierda** sean **menores** o **iguales** a X y todos los que se encuentran a su ***derecha*** sean ***mayores o iguales*** a X.
  - Se repiten los pasos anteriores, pero ahora para los conjuntos de datos que se encuentran a la izquierda y a la derecha de la posición de X en el arreglo.
  - El **proceso termina** cuando **todos** los elementos se encuentran en su **posición correcta** en el arreglo.

QUICKSORT

Sea: 15, 67, 08, 16, 44, 27, 12, 35

Se selecciona un A[i] para X, sea **i=0** para el ejemplo, luego **X=15**

1ra pasada (DER-IZQ)

A[7] >= **15**? 35>=**15**? SI, NO hay intercambio

15	67	08	16	44	27	12	35

A[6] >= **15**? 12>=**15**? **NO**, **HAY** intercambio

12	67	08	16	44	27	15	35

Luego

A: 12, 67, 08, 16, 44, 27, **15**, 35

(IZQ-DER)

A[1] <= **15**? 67<=**15**? **NO**, Si hay intercambio

12	15	08	16	44	27	67	35

Luego:

A:12, **15**, 08, 16, 44, 27, 67, 35

QUICKSORT

A:12, **15**, 08, 16, 44, 27, 67, 35

2da. Pasada (DER-IZQ)

A[5]>=**15**?27>=**15**? SI, No hay intercambio

12	15	08	16	44	27	67	35

A[4]>=**15**?44>=**15**? SI, No hay intercambio

12	15	08	16	44	27	67	35

A[3]>=**15**?16>=**15**? SI, No hay intercambio

12	15	08	16	44	27	67	35

A[2]>=**15**? 8>=**15**? **NO, HAY** intercambio

12	8	15	16	44	27	67	35

Luego:

A: 12, 8, **15**, 16, 44, 27, 67, 35

QUICKSORT

A: 12, 8, **15**, 16, 44, 27, 67, 35

- Como el recorrido de **izquierda a derecha** debería iniciarse en la misma posición donde se encuentra el elemento x, el proceso se termina ya que el elemento **X**, se encuentra en la **posición correcta.**
- Ahora se consideran **2 listas** por ordenar, la que se encuentra a la **izquierda de X** y la que se encuentra a la **derecha de X**

Luego

A: 12, 08, **15**, 16, 44, 27, 67, 35

1er conjunto 2do conjunto

QUICKSORT

2do Conjunto: 16, 44, 27, 67, 35

X=**16**

Se considera el 2do conjunto. El nuevo pivote será X=**16**

1ra pasada (DER-IZQ)

- A[7]>=**16**? → 35>=**16**? SI, No hay intercambio

16	44	27	67	35

- A[6]>=**16**? → 67>=**16**? SI, No hay intercambio

16	44	27	67	35

- A[5]>=**16**? → 27>=**16**? SI, No hay intercambio

16	44	27	67	35

- A[4]>=**16**? → 44>=**16**? SI, No hay intercambio

16	44	27	67	35

Luego

A: **16**, 44, 27, 67, 35

QUICKSORT

2do Conjunto: ~~44~~, 27, 67, 35

X=**44**

Se considera el 2do conjunto.
El nuevo pivote será X=**44**

1ra pasada (DER-IZQ)

A[7]>=**44**? → 35>=**44**? NO, Si hay intercambio

35	27	67	44

Luego

A: 35, 27, 67, **44**

(IZQ-DER)

A[5] <= **44**? 27<=**44**? SI, No hay intercambio

35	27	67	44

A[6]<=**44**? 67<=**44**? **NO**, Si hay intercambio

35	27	44	67

Luego

A: 35, 27, **44**, 67

1er conjunto 2do conjunto

QUICKSORT

2do Conjunto: 67

Un único elemento, no hace falta correr el algoritmo

1er Conjunto: 35, 27 X=**35**

1ra pasada (DER-IZQ)

- A[4]>=**35**? → 27>=**35**? NO, Si hay intercambio

27	35

Luego

A: 27, **35**

De esta manera se logra el **ordenamiento del 2do conjunto** respecto del primer pivote (15), es decir:

16, 27, 35, 44, 67

QUICKSORT

1er Conjunto: 12, 8 X=**12**

1ra pasada (DER-IZQ)

A[1]>=**12**? → 8>=**12**? NO, Si hay intercambio

8	12

Luego

A: 8, **12**

De esta manera se logra el **ordenamiento del 1er conjunto** respecto del primer pivote (15), es decir:

8, 12

QUICKSORT

- El arreglo quedó ordenado
  - 8, 12, 15, 16, 27, 35, 44, 67

- Resumen del Proceso:

A: 15 67 08 16 44 27 12 35

A: 12 08 15 16 44 27 67 35

1er. Conjunto 2do. Conjunto

A: 12 08 15 16 44 27 67 35

A: 12 08 15 16 35 27 44 67

A: 12 08 15 16 27 35 44 67

A: 08 12 15 16 27 35 44 67

QUICKSORT: Análisis de eficiencia

- Es el método **más rápido** de ordenación interna.
- Si se escoge en cada pasada el elemento central del conjunto de datos a analizar, el número de pasadas necesarias para ordenarlo es del orden de **log n**.
- El número de comparaciones promedio se calcula como:

$$C=(n-1)*\log n$$

- Puesto que encontrar el elemento central del conjunto de datos es una tarea difícil, se recomienda que el elemento **se escoja arbitrariamente**.
- El **peor caso** sucede cuando el arreglo ya está ordenado o cuando está en orden inverso y el número de comparaciones máximo será:

$$C_{máx}=(n^2+n)/2-1$$

- El **tiempo promedio de ejecución** es $O(n*\log n)$ y para el **peor caso** es $O(n^2)$.

That's all Folks!